

N100 Financial Intelligence Platform

Analyst Guide

Operational guide covering data, analytics, screening, valuation, clustering, reports, REST API, QA and final validation.

1. Platform Overview

Purpose

N100 Financial Intelligence is a financial-data and analytics platform built around a SQLite database, Python analytics modules, generated reports, and a FastAPI REST interface.

Architecture

The main layers are ETL and validation, financial analytics, screener and peer analysis, NLP support, report generation, REST API, and the existing dashboard.

Current universe

The validated working universe contains 92 companies. Generated tearsheets, valuation summaries, cluster outputs, and API responses are based on this database snapshot.

2. Data Pipeline & Database

Pipeline

Source datasets are loaded, normalized, cleaned, validated, and persisted into db/nifty100.db. Downstream analytics query the database rather than independently maintaining separate financial datasets.

Core tables

Important tables include companies, profitandloss, balancesheet, cashflow, financial_ratios, market_cap, stock_prices, peer_groups, peer_percentiles, sectors, documents, and prosandcons.

Analyst guidance

Annual and TTM records should be distinguished. Missing values are not automatically zero. Sector classification is important for relative analysis.

3. KPI & Ratio Engine

Core metrics

The ratio layer includes net profit margin, operating profit margin, ROE, ROCE, debt-to-equity, interest coverage, asset turnover, revenue/PAT/EPS CAGR, free cash flow, dividend payout and per-share metrics.

Interpretation

Ratios should be interpreted together. High ROE can arise from strong economics or leverage, while high growth can reflect a small historical base.

Quality checks

Extreme values should be investigated against the underlying annual history before being treated as meaningful investment signals.

4. Screener

Presets

Six analyst-editable presets are configured: `quality_compounder`, `value_pick`, `growth_accelerator`, `dividend_champion`, `debt_free_blue_chip`, and `turnaround_watch`.

Quality compounder

The preset combines ROE, debt-to-equity, free cash flow and five-year revenue CAGR thresholds. The validated API response currently contains 21 companies.

API behavior

Missing pandas values are converted to JSON null values at the API boundary without altering the underlying analytical calculations.

5. Peer Analysis

Method

Authoritative peer-group mappings are used to calculate percentile rankings for configured financial metrics. Results are persisted in `peer_percentiles` for efficient API retrieval.

Usage

Peer percentiles should be interpreted within the named peer group and metric. Relative rankings are more meaningful when businesses have comparable economics.

Example

The TCS peer endpoint was validated successfully and returned persisted IT Services peer records.

6. Valuation

Outputs

Valuation artifacts include valuation_summary.xlsx and valuation_flags.csv. The valuation engine supports P/E, P/B, FCF yield, sector-median P/E and five-year median P/E analysis.

Interpretation

Valuation flags are decision-support signals, not recommendations. Low multiples can reflect undervaluation, cyclicalities, leverage or weak business quality.

API

The REST API exposes both valuation flags and the generated valuation summary. The summary was validated at 92 company records.

7. Cash Flow & Capital Allocation

Cash-flow intelligence

Cash-flow analytics examine operating, investing and financing activity and derive indicators for earnings quality and financial resilience.

Free cash flow

FCF should be evaluated with operating cash generation and capital expenditure. Negative FCF is contextual rather than automatically a failure condition.

Capital allocation

Capital-allocation outputs summarize distribution and pattern changes across periods, supporting analysis of dividends, reinvestment and financing behavior.

8. Clustering & Portfolio Analysis

Clustering

KMeans clustering uses normalized financial features with sector-median imputation for missing feature values. The current model uses five clusters.

Current distribution

The validated distribution is 64 Core / Balanced Businesses, 13 Highly Leveraged, 12 High-Margin Growth, 2 Extreme-ROE Businesses, and 1 Exceptional Growth / Base Effect.

Outliers

Cluster outlier analysis highlights extreme ROE, leverage, growth and other feature observations. Extreme values should be checked against historical data and business context.

9. Reports & Tear sheets

Company reports

The platform generated 92 company tearsheets successfully and validated all 92 PDFs. They contain company identity, sector information, headline KPIs, financial history, charts, balance-sheet information and analyst context.

Portfolio summary

The portfolio summary contains one page per company in alphabetical ticker order with company name, sector, top KPIs and trend indicators.

Artifacts

Company PDFs are stored under output/tearsheets. Batch generation evidence is retained in the tearsheet generation report and summary JSON.

10. NLP & Analyst Content

Components

The NLP layer contains parsing and pros/cons generation modules. Outputs include `analysis_parsed.csv`, `parse_failures.csv` and `pros_cons_generated.csv`.

Usage

Generated narrative is analyst-supporting content. Parse failures are retained for review, and generated pros/cons should be checked against the underlying financial data before publication.

11. REST API

Application

The FastAPI application is defined in `src/api/main.py` and mounts `health`, `company`, `screener`, `sector`, `peer`, `valuation`, `portfolio` and `document` routers.

Coverage

Validated endpoints include company profile and financial histories, screener presets, sectors, peers, valuation, portfolio statistics/clusters and tearsheet documents.

Documentation

The machine-readable OpenAPI specification is exported to `docs/openapi.json`. Missing resources return HTTP 404 responses.

12. Dashboard Workflow

Recommended workflow

Start with company context, use the screener to narrow candidates, compare peers, inspect trends and cash flow, review valuation, and finish with the company tearsheet.

Consistency

Dashboard and API consumers should use the same database-backed analytical outputs. Generated files should not be manually modified as a substitute for updating the underlying pipeline.

13. Data Quality & Validation

Validation

The project includes ETL validation, ratio validation, report validation, duplicate checks, company-count checks and explicit cluster/report verification.

Current evidence

92 companies are represented; cluster labels contain 92 unique assignments; valuation summary contains 92 records; tearsheet generation and PDF validation both achieved 92/92.

Anomalies

Some ratios are extreme. These observations are surfaced for review and may reflect sector structure, accounting relationships or base effects rather than implementation errors.

14. Testing & Performance

Automated tests

The project test suite completed with 189 passing tests. API regression tests increased the full suite to 210 passing tests with zero failures.

API coverage

Tests cover health, companies, screener, sectors, peers, valuation, portfolio and document endpoints.

Performance

Ten concurrent requests to the `quality_compounder` screener endpoint all returned HTTP 200. Total wall time was 2.589 seconds, average 2.472 seconds, maximum 2.585 seconds, comfortably below the 10-second target.

15. Analyst Interpretation & Limitations

Interpretation rules

Use multiple metrics together, compare appropriate peers, investigate extreme values, distinguish missing values from zero, and treat screeners, flags and clusters as decision-support tools.

Limitations

Results depend on source-data coverage and quality. Missing classifications or historical fields can affect downstream analysis. Cluster labels are model-derived descriptions rather than permanent economic classifications.

Snapshot principle

API and report outputs represent the current database and generated-artifact snapshot.

16. Operations & Final Checklist

Routine commands

Use the project's Python modules for analytics/report generation, Uvicorn for FastAPI, `pytest -q` for the full test suite, and `docs/openapi.json` as the machine-readable API contract.

Final evidence

Before sign-off, verify database counts, generated reports, cluster assignments, automated tests, API health, API performance, documentation artifacts and acceptance-gate results.

Current status

At this stage the project has 210 passing tests, validated REST endpoints, 92 generated and validated tearsheets, five validated clusters, and a passing concurrent API performance test.